Analytics Backend Coding Guidelines

1. Core Framework & Architecture

- Python ≥ 3.12
- FastAPI ≥ 0.114.0
- Pydantic ≥ 2.x
- Postgresql ≥ 17
- SQLalchemy ≥ 2.x
- Xlsxwriter >= 3.x
- Pandas >= 2.x
- Pypika >= 0.48
- Redis >=7.1.0
- Celery>=5.6.2
- Redshift_connector >= 2.x
- Adopt 3-tier/MVC: routers (endpoints) → services → data access (repositories)
- Adopt Modular architecture "api/, models/, services/, repositories/, schemas/, tests/"
- Adopt Django like app structure with base core app with shared utilities, each app represent specific domain (Modular Monolith Architecture - Inspired from Domain Driven Design)
- Comply with ECLAT Backend Coding Standards
- SQL Query generation with SQL Builder and Query Builder for complex filters, search, sort etc.
- Use async/await; avoid blocking calls in endpoints

2. Dependency & Version Management

- Use UV for dependency & lockfile management
- Pin version ranges realistically

3. Code Quality & Static Analysis

- Format: Ruff
- Lint: Ruff
- Type Hints: Use type hints throughout the codebase for clarity and to enable static analysis
- Typing: MyPy
- Pre-commit hooks: auto-lint, test, type-check

4. Testing

- Use pytest for unit, integration, HTTP tests

5. Docker & Containerization

- Base image: python:3.12.11-alpine3.21
- Docker builds: non-root user for dev and prod
- Docker Compose: development environment with service dependencies

6. Security & Configuration

- Use Pydantic BaseSettings with `.env`, `.env.example` (Uses Parameter Store on deployed instances)
- Auth: OAuth2/JWT + RBAC (Will be replaced once Centralized SSO Service ready)
- Secrets Management: Never hardcode secrets; use environment variables or secret managers
- HTTPS enforcement, input validation
- Audit logging via middleware + DB tables

7. Database & Migrations

- Use PostgreSQL as Transactional Database
- Use async SQLAlchemy as ORM and Alembic as migration tool
- Implement Connection pooling
- UUID for uniquely identifying the database records
- Use the redshift-connector with connection pooling for communication with the Redshift database.

8. Logging

- Structured Custom logging configuration → stdout

9. Deployment

- ASGI server (Uvicorn/Gunicorn with Uvicorn workers)
- Workers ≈ 2×CPU cores + 1

10. Advanced Features

- Compliance: HIPAA
- Caching & rate-limit layers (Redis, proxies)
- Task queuing via Celery

11. FastAPI Specific Practices

- Pydantic Models: Use Pydantic models for all request and response schemas to ensure robust data validation and serialization

- Async Endpoints: Use async def for I/O-bound endpoints to leverage FastAPI's asynchronous capabilities
- Dependency Injection: Use FastAPI's Depends for injecting dependencies and managing resources
- RESTful Design: Follow RESTful conventions for endpoint naming and HTTP method usage
- Automatic Documentation: Leverage FastAPI's auto-generated OpenAPI docs by providing type hints and docstrings
- Input Validation: Rely on Pydantic for input validation, and always validate user input at the API boundaries
- Connection Management: Use dependency injection to manage database sessions and connections

12. Documentation

- Inline Documentation: Use docstrings for all public classes, methods, and functions
- API Docs: Maintain up-to-date OpenAPI/Swagger documentation via FastAPI's built-in support
- README & Onboarding: Provide a comprehensive README.md with setup instructions, coding standards, and contribution guidelines